

Compiling Ancient Greek Semantic DAGs from Universal Dependencies

Group 7

Evan Liu, George Jiang, Bennet Wang, and Haocheng Han
New York University

May 2026

Abstract

This paper presents a compiler that turns morphosyntactic analyses from Ancient Greek Universal Dependencies (UD) treebanks into explicit, inspectable, and structurally valid semantic graphs. Each sentence is converted into a directed acyclic graph (DAG) whose nodes preserve token, lemma, part of speech, and morphology, while edges encode a compact semantic inventory: AGENT, THEME, MODIFIER, COMPLEMENT, and COORD. I evaluate the system with three complementary kinds of evidence. First, on a manually labeled historical gold set of 21 real PROIEL and Perseus sentences (236 tokens, 122 gold semantic edges), the complete graph achieves 68.82 precision, 95.90 recall, and 80.14 F1, compared with 19.55 F1 for a local baseline that ignores UD dependency edges and predicts semantic relations only from token order, POS, and case heuristics. Excluding MODIFIER edges raises the score on core semantic roles to 91.75 precision, 94.68 recall, and 93.19 F1, compared with 22.56 F1 for the same baseline. Second, on the full combined Ancient Greek UD test material in the repository (2,353 sentences, 31,913 tokens), the compiler preserves all nodes, produces valid DAGs for every sentence, maps 79.36% of all non-punctuation dependencies, covers 98.98% of content dependencies, and covers 99.96% of predicate-argument dependencies. Third, a supporting raw-pipeline report shows that adding a local UD-derived corpus lexicon raises lemma accuracy from 0.2478 to 0.9832, UPOS accuracy from 0.0308 to 0.9378, proxy semantic-type accuracy from 0.0667 to 0.9409, and relation F1 from 0.0626 to 0.2992 on 200 real UD sentences. The main limitations are semantic over-generation of modifier edges and, for the local lexicon result, possible overlap between the UD-derived lookup table and evaluation data. These results show that UD-backed graph compilation can recover a strong semantic role structure for Ancient Greek while preserving transparent graph structure and full DAG validity.

1 Introduction

Ancient Greek texts are often accessed through translations, commentaries, and lexical notes, but many research questions require explicit structure over the original language. A classic example is determining who did what to whom, under which conditions, and with what modifiers. Dependency treebanks already provide valuable morphosyntactic information for Ancient Greek, but their labels remain primarily syntactic. The central question is whether those syntactic analyses can be compiled into a more semantic, graph-based layer that remains easy to inspect and evaluate.

The system converts UD-annotated sentences into semantic directed acyclic graphs (DAGs). Each token becomes one node. Each node preserves the original token surface form, lemma, part of speech, and morphological features. Syntactic dependency labels are then mapped into a smaller semantic edge inventory: AGENT, THEME, MODIFIER, COMPLEMENT, and COORD. The output is

structurally validated with graph checks so that the representation is not only interpretable, but also well formed.

The paper makes five concrete contributions: a compact semantic DAG representation grounded in real treebank inputs, a compiler from UD annotations to that graph inventory, a small manual evaluation set of historical Greek sentences labeled directly in the target format, a local UD-derived lexicon for the raw analyzer, and an empirical evaluation that combines baseline comparison, corpus-scale coverage statistics, raw-pipeline proxy metrics, and smoke-test records.

2 Related Work

This work sits at the intersection of dependency annotation, classical treebanking, semantic role labeling, and semantic graph representations.

Universal Dependencies provides a cross-linguistically consistent framework for tokenization, part-of-speech tagging, morphological features, and dependency relations [7]. In this setting, UD matters because it supplies a stable annotation layer from which semantic abstraction can begin. This makes it possible to reorganize syntactic structure into semantic structure while preserving the original token-level linguistic information.

For Ancient Greek specifically, existing dependency treebanks make this kind of work possible. The Ancient Greek and Latin Dependency Treebanks described by Bamman and Crane [1] demonstrate the value of large, manually annotated classical corpora for linguistic and philological research. The PROIEL project [6] extends the treebanking tradition to parallel and historical Indo-European texts, including Ancient Greek material that is already distributed in UD form. These resources provide the morphosyntactic basis for the system input used here.

At the semantic level, the most relevant precedent is semantic role labeling. PropBank-style annotation formalizes predicate-argument structure over events and their participants [8]. The system adopts the same general intuition that subject, object, and clausal dependencies can often be reorganized into semantically meaningful role edges.

The approach is also inspired by graph-based semantic representations such as Abstract Meaning Representation (AMR), which represents sentence meaning as a graph over events, entities, and relations [2]. This paper takes the same graph-oriented perspective while using a compact inventory of node types and role labels tailored to Ancient Greek treebank data.

What distinguishes the present system from the work above is its combination of Ancient Greek UD input, a deliberately small semantic label inventory, explicit graph validation, and evaluation directly on manually labeled historical Greek sentences.

3 Task Definition and Data

3.1 Task Definition

The system input is a sentence with UD-style token-level annotations: surface tokens, lemmas, part-of-speech tags, morphological features, and dependency links. The output is a semantic DAG in which every token is preserved as a node and every retained relation is rewritten into one of five target labels: AGENT, THEME, MODIFIER, COMPLEMENT, or COORD. The task is to compile these token-level annotations into a semantic graph.

The graph is intended to support downstream interpretability. A reader should be able to inspect the output and recover the main event structure, participant roles, modifier attachments,

and clausal relationships without consulting the original dependency labels. At the same time, the graph should remain structurally valid: acyclic, internally consistent, and free of malformed edges.

3.2 Treebank Inputs

The repository includes two public Ancient Greek UD test files used in the experiments:

- `data/real_eval/grc_perseus-ud-test.conllu`
- `data/real_eval/grc_proiel-ud-test.conllu`

When converted with the compiler, these files yield 2,353 sentences and 31,913 non-punctuation tokens. These treebanks provide the morphosyntactic analyses from which the semantic DAGs are constructed.

3.3 Manual Historical Semantic Gold

The main semantic evaluation uses `data/gold_semantic/historical_semantic_dags.jsonl`. This file contains 21 manually labeled historical Greek sentences selected from the PROIEL and Perseus UD test materials. The examples total 236 tokens and 122 gold semantic edges in the target inventory.

This manual gold set is intentionally content-semantic. It emphasizes event participants, complements, coordination, and semantically relevant modifiers. Articles, discourse particles, and other purely functional markers are not systematically rewarded. That choice is important because a naive syntax-to-graph mapping could otherwise inflate scores by reproducing formal syntactic detail that does not contribute much to semantic interpretation.

The repository also evaluates a second view of the same manual gold set that removes MODIFIER from scoring. In that setting, the remaining core roles (AGENT, THEME, COMPLEMENT, and COORD) account for 94 gold edges. This core-role subset helps isolate whether the system’s main semantic contribution lies in predicate-argument structure or in broad syntax-to-modifier projection.

3.4 Local UD Lexicon Report

A supporting project artifact, `presentation/ud_lexicon_report.html`, records a raw-analyzer experiment in which local UD CoNLL-U files are converted into a corpus-backed lookup table [4]. The added module, `src/corpus_lexicon.py`, reads `data/real_eval/*.conllu` and maps normalized Greek surface forms to lemma, UPOS, and morphology. The report is not a replacement for the manual historical semantic gold evaluation. It is a diagnostic record showing how much token-level analysis improves when the raw pipeline can consult locally observed UD lexical evidence.

The report’s caveat is important: because the lexicon is built from local UD data, a stricter future experiment should build the lookup table from a training partition and evaluate only on held-out sentences. I therefore treat the report as supporting evidence for the value of corpus-backed lexical hints, not as a final generalization claim.

4 Methodology

4.1 Node Construction

The compiler creates one graph node per retained linguistic token. Each node stores the original surface form, lemma, UD part of speech, morphological features, a semantic type, a semantic

Table 1: Main mapping from UD dependency labels to semantic DAG labels.

UD labels	DAG label	Intuition
nsubj, csubj, obl:agent	AGENT	canonical subjects and agentive obliques
obj, iobj, nsubj:pass, csubj:pass, obl:arg	THEME	objects and patient-like arguments
xcomp, ccomp, acl, advcl, advcl:cmp, obl	COMPLEMENT	clausal and oblique complements
amod, nmod, nummod, advmod, appos, det	MODIFIER	attributive, adverbial, or dependent modifiers
conj	COORD	coordination between linked conjuncts

role, and a confidence score. The semantic typing scheme is intentionally compact. Nouns and proper nouns typically become *entity* nodes, verbs and auxiliaries become *event* nodes, adjectives become *property* nodes, adverbs become *manner* nodes, and conjunction or linker-like items become *connector* nodes. This inventory is designed to support graph inspection, role attachment, and evaluation across real historical sentences.

The compiler copies lemma and morphology information directly into the node representation. This keeps the graph closely aligned with the source treebank annotations and preserves the linguistic details needed for semantic analysis.

4.2 Corpus-Backed Raw Lexical Hints

The raw analyzer is weaker than the UD-backed compiler because it does not receive gold dependency edges, and in the original version it also had sparse lexical coverage. The local corpus lexicon addresses the lexical part of that problem. It indexes observed CoNLL-U forms by a normalized surface key and stores the most frequent lemma, UPOS tag, and morphological feature bundle for each form. Exact matches are used first; suffix-derived fallback rules provide an interpretable guess for unseen inflected forms.

The lexicon is used during lexeme normalization, semantic typing, and relation prediction. Lexeme normalization can recover a UD lemma instead of emitting an opaque fallback root. Semantic typing can use UPOS defaults such as NOUN to entity, VERB to event, ADJ to property, and ADV to manner. Relation prediction can use morphology, especially Case, and can skip function-word categories whose surface syntax should not become a content edge. The raw analyzer still keeps its transparent ambiguity fallback: known lemma candidates are ranked with character n-gram TF-IDF cosine similarity and a string ratio, implemented with scikit-learn and Python’s standard library [10, 9]. The graph validator uses `networkx` for cycle checks [5]. Earlier noisy runtime tokenizer-download behavior was removed from the recorded pipeline so that evaluation depends on local resources rather than hidden network state; NLTK remains part of the tokenizer stack [3].

4.3 Dependency-to-Semantic Mapping

The core of the system is a rule-based mapping from UD dependency labels into the target semantic edge inventory. Table 1 summarizes the main mapping decisions.

The mapping is intentionally conservative in its label inventory and broad in coverage. Many

syntactic distinctions are collapsed into a small number of readable semantic edge types. The trade-off is that some syntactically valid relations, especially modifier-like ones, may become semantically too broad once projected into the smaller label space.

Several UD relations are intentionally not treated as semantic edges. In the corpus-scale coverage analysis, the largest unmapped relation types are `case`, `cc`, `discourse`, `mark`, and `cop`. These are mostly function markers or structural helpers rather than the central content relations the graph is meant to expose. Excluding them makes the graph more semantically readable and explains why content-dependency coverage is much higher than overall dependency coverage.

4.4 Graph Building and Validation

After node and edge creation, the compiler builds a directed graph with stable node identifiers such as `n0`, `n1`, and `n2`. Duplicate edges, self-loops, and malformed links are removed before validation. Validation checks confirm that edge labels come from the allowed inventory, endpoints are valid, and the final graph is acyclic. The implementation also verifies predicate-argument well-formedness for required structures and uses `networkx` for cycle detection.

These checks make the output usable as structured data, not just as a readable annotation. High local edge accuracy is not enough if the graph as a whole is invalid.

5 Experiments

5.1 Experiment 1: Manual Historical Gold Evaluation

The first experiment measures exact edge precision, recall, and F1 on the 21 manually labeled historical sentences. A predicted edge is counted as correct only if its source token, destination token, and label all match the gold edge. This metric is strict but appropriate because the output is a graph, not a bag of independent labels. The experiment also reports DAG validity rate.

Because the system is deterministic and rule-based, no supervised model is trained and no statistical parameters are tuned. The public UD test files and manual historical gold set are used only for reported evaluation. Development work consisted of designing the dependency-to-semantic mapping and validating the graph construction procedure, rather than fitting rules to a train/dev split.

Two systems are compared on this task. The first is the full compiler described above, which maps UD dependencies directly into semantic graph edges. The second is a deliberately simple baseline. The baseline uses the same UD tokens, lemmas, POS tags, and morphological features, but it ignores UD dependency links. It instead predicts semantic edges from local word order, POS, and case heuristics using the repository’s existing linear relation predictor. The comparison asks how much the dependency layer contributes beyond a simple local heuristic.

For both systems, two scores are reported. The *full graph* score evaluates all five labels. The *core roles* score excludes MODIFIER and evaluates only AGENT, THEME, COMPLEMENT, and COORD. This split is motivated by early inspection of the outputs: the compiler appears strong on event structure but less selective about which syntactic dependents deserve a semantic modifier edge.

5.2 Experiment 2: Corpus-Scale UD Conversion

The second experiment measures how the compiler behaves when applied to the full combined Ancient Greek UD test data in the repository. Here the focus is on coverage and robustness over the full converted corpus.

The relevant metrics are:

- overall dependency coverage: the proportion of non-root, non-punctuation UD dependencies that receive a semantic edge;
- content dependency coverage: the same calculation, but excluding relations such as `case`, `cc`, `mark`, `cop`, `discourse`, `aux`, `fixed`, and `flat:name`;
- predicate-argument coverage: the proportion of subject, object, and argument-like dependencies that are converted into semantic edges;
- DAG validity rate: the proportion of converted sentences whose final graphs pass structural validation.

This experiment asks whether the system scales cleanly to thousands of real treebank sentences while preserving the main semantic structure.

5.3 Experiment 3: Raw UD Lexicon and Smoke Checks

The third experiment is documented in `presentation/ud_lexicon_report.html`. It compares the same UD-proxy evaluator before and after adding the local UD-derived lexicon to the raw analyzer. The evaluator reconstructs raw input from UD token forms, runs the project pipeline, then compares predicted lemmas, UPOS tags, proxy semantic types, and selected mapped relation edges against UD-derived gold labels.

This experiment is useful because it measures the proposal’s raw-text pipeline rather than the parser-backed UD compiler. It also exposes the internal error shape: if lemma, UPOS, and semantic type scores are high but relation F1 remains low, the bottleneck is relation induction rather than token-level linguistic normalization.

I also ran smoke checks over the repository’s smaller gold sets and direct metric utilities. These checks include dev/test evaluation, the simple manual semantic DAG set, the historical semantic DAG set, the `Metrics` and `BlockMetrics` helper classes, PRF and labelled-F1 helpers, DAG validity, and the TF-IDF similarity fallback. These smoke checks are not a broad benchmark; they are regression evidence that the scoring utilities and project entry points still behave as expected after the merge and report update.

5.4 Reproducibility

The main experimental entry points in the repository are `scripts/run_ud_dag.py` for corpus-scale conversion, `scripts/evaluate_historical_gold.py` for manual semantic evaluation, `scripts/evaluate_ud.py` for the raw UD-proxy check, and the local HTML report `presentation/ud_lexicon_report.html` for the before/after lexicon command record. All quantitative results reported below were regenerated locally from those scripts inside the cloned project workspace. The project code, data files, evaluation scripts, and paper sources are available at https://github.com/Eliguli712/Greek_Learning.

6 Results

Table 2 shows the main comparison between the full compiler and the local baseline on the manual historical gold set.

Table 2: Comparison between the full compiler and the local baseline on 21 Ancient Greek historical sentences.

System and setting	Precision	Recall	F1	Gold edges
Baseline, full graph	18.06	21.31	19.55	122
Compiler, full graph	68.82	95.90	80.14	122
Baseline, core roles only	21.78	23.40	22.56	94
Compiler, core roles only	91.75	94.68	93.19	94

Table 3: Per-label results for the full compiler on the manual historical gold set.

Label	Precision	Recall	F1	Gold edges
AGENT	80.95	100.00	89.47	17
THEME	93.33	90.32	91.80	31
MODIFIER	38.36	100.00	55.45	28
COMPLEMENT	94.59	94.59	94.59	37
COORD	100.00	100.00	100.00	9

The comparison is decisive. On the full graph, the compiler improves from 19.55 F1 to 80.14 F1. On core semantic roles, it improves from 22.56 F1 to 93.19 F1. This gap is large enough that it cannot be explained by minor tuning choices. It shows that the UD dependency layer is doing real work for the task, especially for event structure and clausal attachment.

The baseline fails most clearly on complements. Because it ignores dependency links and relies only on local order and morphology, it never recovers the gold COMPLEMENT edges in this evaluation. It also performs weakly on AGENT and THEME, and only modestly on COORD. This is exactly the behavior one would expect from a local heuristic in a language with richer morphology and less rigid word order than English.

Table 3 breaks down the full compiler’s per-label results on the same historical evaluation set.

The strongest result is the compiler’s performance on the semantic backbone of the sentence. Core roles reach 93.19 F1, with particularly strong performance on COMPLEMENT and perfect performance on COORD in this sample. Both AGENT and THEME also score above 89 F1. These numbers support the claim that the system is capturing predicate-argument structure reliably once strong upstream syntax is available.

The weaker result is the full-graph score of 80.14 F1. The gap between full graph and core roles comes almost entirely from MODIFIER. Modifier recall is 100%, which means the compiler almost never misses a gold semantic modifier once one exists. However, modifier precision is only 38.36%. In other words, the system attaches too many dependents as semantic modifiers. This is consistent with the system design: a broad set of UD syntactic modifiers is collapsed into one semantic label, so purely structural or lightly semantic dependents are often over-produced.

Every evaluated historical sentence still yields a valid DAG. The validity rate is 100%, which means the structural layer of the system is robust even when the semantic edge inventory is imperfect.

Table 4 presents the corpus-scale conversion results on the full combined Ancient Greek UD test material.

These coverage results sharpen the interpretation of the manual gold numbers. The system’s lower full-graph precision is not caused by a failure to preserve predicate-argument structure. In

Table 4: Corpus-scale conversion statistics on the combined Ancient Greek UD test files in the repository.

Metric	Value	Notes
Sentences converted	2,353	Perseus + PROIEL test material
Tokens converted	31,913	non-punctuation tokens
Mapped dependencies	23,458	semantic edges produced
Unmapped dependencies	6,102	mostly function markers
Overall dependency coverage	79.36%	all non-root, non-punctuation deps
Content dependency coverage	98.98%	excluding function-marker relations
Predicate-argument coverage	99.96%	subject/object/argument-style deps
DAG validity rate	100.00%	all converted graphs valid

Table 5: Raw analyzer UD-proxy results from the local UD lexicon report on 200 real UD sentences.

Metric	Before lexicon	After lexicon	Absolute gain
Lemma accuracy	0.2478	0.9832	+0.7354
UPOS accuracy	0.0308	0.9378	+0.9070
Proxy semantic-type accuracy	0.0667	0.9409	+0.8742
Mapped relation F1	0.0626	0.2992	+0.2366

fact, predicate-argument coverage is essentially complete at 99.96%. Instead, the remaining gap comes from exactly the kinds of dependencies the compiler treats cautiously: function markers and broad modifier-like relations.

The unmapped dependency counts support that interpretation. The largest unmapped categories are **case** (2,322), **cc** (1,660), **discourse** (788), **mark** (580), and **cop** (490). These are mostly structural or functional relations rather than the main semantic relations the system is trying to expose. Once such categories are excluded, content dependency coverage rises to 98.98%.

Table 5 shows that the local UD lexicon changes the raw analyzer most strongly at the token-linguistic layers. Lemma accuracy rises by 0.7354, UPOS accuracy by 0.9070, and proxy semantic-type accuracy by 0.8742. This is the expected effect of a corpus-backed lookup table: once the analyzer can identify observed forms, it no longer has to infer lemma and POS from sparse hand-written rules. Relation F1 also improves, from 0.0626 to 0.2992, but it remains much lower than the lexical scores. This confirms the main diagnostic lesson from the experiments: relation prediction is the hardest part of the raw pipeline.

The smoke checks in Table 6 support the reliability of the reported evaluation machinery. The toy dev/test scores are perfect because those sets are small and controlled, so they should not be over-interpreted as corpus performance. The simple manual semantic DAG set is also small but useful as a transparent correctness check. The historical semantic DAG score is the more realistic manual-gold signal, and its 0.8014 F1 matches the modifier over-generation pattern discussed above. Finally, the direct property checks show that the metric implementation itself is internally consistent.

Table 6: Smoke-check record after merging the metrics helpers and report artifacts.

Check	Recorded result
Dev gold evaluation	lemma accuracy 1.0; semantic macro/micro F1 1.0; relation P/R/F1 1.0/1.0/1.0; DAG validity 1.0
Test gold evaluation	same 1.0 scores across lemma, semantic F1, relation F1, and DAG validity
Simple manual semantic DAG evaluation	relation P/R/F1 0.9583/1.0/0.9787; DAG validity 1.0
Historical semantic DAG evaluation	relation P/R/F1 0.6882/0.9590/0.8014; DAG validity 1.0
Direct property smoke	Metrics , BlockMetrics , PRF helpers, labelled F1, accuracy, DAG validity, and TF-IDF similarity all passed assertions
TF-IDF fallback	transliterated <i>logou</i> matched <i>logos</i> with cosine 0.8306 and blended score 0.8183

7 Analysis and Evaluation

At its core, the paper addresses a clear technical problem: compiling an existing morphosyntactic analysis into a semantic graph for Ancient Greek. The evaluation shows that this formulation supports strong predicate-argument recovery, transparent graph structure, and reliable large-scale conversion across real treebank data.

The experimental record supports a layered evaluation rather than a single headline score. The UD-backed compiler is strong when dependency structure is available: core semantic roles reach 93.19 F1, every evaluated graph is valid, and predicate-argument coverage is essentially complete at corpus scale. The raw pipeline improves sharply after adding the local corpus lexicon, especially on lemma, UPOS, and semantic-type accuracy. However, relation F1 remains much lower in the raw UD-proxy setting. The correct evaluation is therefore not “the raw system solves Ancient Greek semantics,” but rather: the representation and UD-backed compiler are strong, the raw lexical and semantic-token layers are now usable with local corpus support, and raw relation induction remains the main research bottleneck.

The strongest outcome is that a compact semantic label inventory is enough to recover the predicate-argument backbone of real historical Greek sentences. The manual evaluation shows this directly, and the corpus-scale coverage analysis supports it from another angle. The graph representation is therefore a useful way to expose role structure, clausal structure, and modifier attachment in a form that remains easy to inspect and validate.

The main weakness is modifier over-generation. Several design decisions contribute to this. First, the mapping groups many syntactically different UD relations under one semantic label. Second, the historical gold deliberately omits many function-like or low-content modifiers, so a syntactically faithful projection can look semantically noisy. Third, Ancient Greek often contains particles, determiners, and other dependents whose semantic contribution is subtle or context dependent. A single MODIFIER label is too coarse to handle all of these cases well.

The main limitations extend beyond modifiers. The manual semantic gold set is small, and the system inherits the quality of the upstream UD annotations used as input. Even so, the results are strong and consistent. The baseline comparison shows that the final compiler captures much more

of the semantic structure than a local heuristic, and the main remaining error source is precise semantic filtering of modifiers rather than event structure itself. These results should not be read as a direct state-of-the-art comparison with full Ancient Greek parsers, semantic role labelers, or AMR-style systems. The project instead contributes a reproducible compiler and evaluation setup that could later be paired with stronger upstream parsing or learned semantic filtering.

8 Conclusion and Future Work

The paper presents a semantic DAG compiler for Ancient Greek. The system preserves token, lemma, part of speech, morphology, and syntactic structure, then reorganizes those inputs into a compact semantic graph with explicit role labels. On 21 manually labeled historical sentences, the compiler reaches 80.14 F1 on the full graph and 93.19 F1 on core semantic roles, compared with 19.55 and 22.56 F1 for a local baseline that ignores dependency edges. On the full combined UD test files in the repository, it converts 2,353 sentences with 100% DAG validity and near complete coverage of predicate-argument dependencies. The supporting raw UD-lexicon report shows that corpus-backed lexical hints make the raw analyzer far more reliable at lemma, UPOS, and semantic-type prediction, while still leaving relation F1 as the main unsolved component. The smoke-check record confirms that the local evaluation scripts, metrics utilities, DAG validity checks, and TF-IDF fallback continue to run after the report integration.

Future work should focus on the part of the system that the evaluation exposed as fragile: modifier filtering. One direction is to split MODIFIER into a finer taxonomy that distinguishes semantic attributes, determiners, discourse markers, and syntax-only attachments. Another is to learn a pruning or relabeling layer from manually annotated semantic graphs instead of relying on a single deterministic projection. A third is to expand the manual gold set so that evaluation covers more authors, genres, and clause types. A fourth is to turn the local UD-derived lexicon result into a stricter train/test experiment: build `src/corpus_lexicon.py`'s lookup table from a training partition and evaluate only on held-out sentences. Longer-term work could also integrate richer semantic information, such as valency classes, animacy, or limited coreference, while preserving the graph validity guarantees that made the current system attractive.

The result is a reproducible compiler that turns real Ancient Greek treebank analyses into inspectable graph structure.

Contributions

Evan Liu led the final evaluation and reporting work, including the UD-backed DAG evaluation, historical semantic gold evaluation, baseline comparison, presentation materials, and final paper preparation; he also delivered the final presentation. Haocheng Han contributed the initial project setup, morpheme and lexeme framework, lexicon resources, and early design documentation. Bennet Wang contributed the minimal viable compiler pipeline, phoneme processing, validation/evaluation utilities, and Aristotle test outputs. George Jiang contributed updates to the semantic tokenization component.

References

- [1] David Bamman and Gregory Crane. The ancient greek and latin dependency treebanks. In *Language Technology for Cultural Heritage*, pages 79–98. Springer, 2011. doi: 10.1007/978-3-642-20227-8_5.

- [2] Laura Banarescu, Claire Bonial, Shu Cai, Madalina Georgescu, Kira Griffitt, Ulf Hermjakob, Kevin Knight, Philipp Koehn, Martha Palmer, and Nathan Schneider. Abstract meaning representation for sembanking. In *Proceedings of the 7th Linguistic Annotation Workshop and Interoperability with Discourse*, pages 178–186, 2013.
- [3] Steven Bird, Ewan Klein, and Edward Loper. *Natural Language Processing with Python*. O’Reilly Media, 2009.
- [4] Greek Learning Project. Local UD lexicon report. [../presentation/ud_lexicon_report.html](#), 2026. Local repository report documenting the UD-derived lexicon experiment, command lines, before/after metrics, and evaluation caveat.
- [5] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008.
- [6] Dag T. T. Haug and Marius L. Jøhndal. Creating a parallel treebank of the old indo-european bible translations. In Caroline Sporleder and Kiril Ribarov, editors, *Proceedings of the Second Workshop on Language Technology for Cultural Heritage Data (LaTeCH 2008)*, pages 27–34, 2008.
- [7] Joakim Nivre, Marie-Catherine de Marneffe, Filip Ginter, Yoav Goldberg, Jan Hajič, Christopher D. Manning, Ryan McDonald, Slav Petrov, Sampo Pyysalo, Natalia Silveira, Reut Tsarfaty, and Daniel Zeman. Universal dependencies v1: A multilingual treebank collection. In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC’16)*, pages 1659–1666, 2016.
- [8] Martha Palmer, Daniel Gildea, and Paul Kingsbury. The proposition bank: An annotated corpus of semantic roles. *Computational Linguistics*, 31(1):71–106, 2005. doi: 10.1162/0891201053630264.
- [9] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [10] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988. doi: 10.1016/0306-4573(88)90021-0.